NAME: Emaan Farooq

ROLL NO: 24251652017

BS SEMESTER : 3

SECTION: B

SESSION:2024-28

COURSE:Data Structure & Algorithms

DEPARTMENT: SOFTWARE ENGINEERING
SUBMITTED TO: Mam Tuba

Project

LAHORE COLLEGE FOR WOMEN UNIVERSITY

Data Structures & Algorithm Project

Project Title: Hospital Patient Management System (HPMS)

1. Problem Statement

In hospitals, managing patient information manually is a complex and time-consuming task. Large numbers of patients visit hospitals every day, and their data such as personal details, diseases, and emergency priority must be stored, updated, and retrieved efficiently. Traditional paper-based systems are slow, unreliable, and prone to errors.

The purpose of this project is to develop a computerized Hospital Management System using appropriate data structures. The system will allow hospital staff to add, search, delete, and display patient records in an organized manner. It will also manage emergency patients based on priority and maintain a history of deleted records. All data will be saved permanently using file handling.This project demonstrates how real-world problems can be solved effectively using fundamental data structures.

2. Objectives of the Project

The main objectives of the project are:

- To design a hospital data management system.
- To store and manage patient records efficiently.
- To implement core data structure operations such as insert, search, and delete.
- To manage emergency patients using priority-based processing.
- To maintain the history of deleted records.
- To store and retrieve data from files.
- To provide a user-friendly menu-driven interface.

3. Chosen Data Structures and Justification

This project uses the following THREE major data structures:

A. Linked List – For Patient Records

All patient information is stored using a Linked List.

Data Stored:

- Patient ID
- Name
- Age
- Disease
- Emergency Priority

```python
class PatientNode:
    def __init__(self, pid, name, age, disease, priority):
        self.pid = pid
        self.name = name
        self.age = age
        self.disease = disease
        self.priority = priority  # emergency priority
        self.next = None
class PatientLinkedList:
    def __init__(self):
        self.head = None
    def insert(self, pid, name, age, disease, priority):
        new_node = PatientNode(pid, name, age, disease, priority)
        if self.head is None:
            self.head = new_node
```

Justification:

- Patient records are dynamic and keep changing.
- Linked List allows easy insertion and deletion.
- No fixed size limitation.
- Efficient for sequential storage and display of records.

Operations implemented using Linked List:

- Insert new patient
- Search patient by ID
- Delete patient
- Display all patients sequentially

B. Max Heap – For Emergency Patient Handling

A Max Heap is used to manage emergency patients based on priority.

```python
class MaxHeap:
    def __init__(self):
        self.heap = []
    def insert(self, patient):
        self.heap.append(patient)
        self._heapify_up(len(self.heap) - 1)
    def _heapify_up(self, index):
        parent = (index - 1) // 2
        if index > 0 and self.heap[index].priority > self.heap[parent].priority:
            self.heap[index], self.heap[parent] = self.heap[parent], self.heap[index]
            self._heapify_up(parent)
```

Justification:

- Emergency cases must be handled according to priority.
- Max Heap always keeps the highest priority patient at the top.

- It allows fast retrieval of the most critical patient.
- Suitable for level-order display of emergency cases.

Operations implemented using Max Heap:

- Insert emergency patient
- Delete highest priority patient
- Display emergency patients in level order

C. Stack – For Deleted Patient History

A Stack is used to store the history of deleted patients.

```
class Stack:
    def __init__(self):
        self.stack = []
    def push(self, item):
        self.stack.append(item)
    def display(self):
        if not self.stack:
            print("No deleted patient history.")
            return
        print("\nDeleted Patients History:")
        for p in reversed(self.stack):
            print(f"ID:{p.pid} Name:{p.name}")
def save_data(llist, heap):
    with open("patients.txt", "w") as f:
        temp = llist.head
        while temp:
            f.write(f"{temp.pid},{temp.name},{temp.age},{temp.disease},{temp.priority}\n")
            temp = temp.next
    print("Data saved successfully.")
```

Justification:

- Stack follows LIFO (Last In First Out) principle.
- It keeps track of recently deleted records.
- Useful for maintaining history and possible recovery.

Operations implemented using Stack:

- Push deleted patient record
- Display deleted patient history

4. Functional Requirements of the System

The system provides the following major functionalities:

1. Insert New Patient Record

- User can add a new patient with full details.
- The record is stored in Linked List.
- If a patient has emergency priority, it is also inserted into Max Heap.

```python
if choice == '1':
    pid = input("Patient ID: ")
    name = input("Name: ")
    age = int(input("Age: "))
    disease = input("Disease: ")
    priority = int(input("Emergency Priority (1-10): "))
    llist.insert(pid, name, age, disease, priority)
    heap.insert(PatientNode(pid, name, age, disease, priority))
    print("Patient added successfully.")
```

2. Search Patient Record

- User can search a patient using Patient ID.
- Linked List search is used to locate the record.

```python
elif choice == '2':
    pid = input("Enter Patient ID to search: ")
    patient = llist.search(pid)
    if patient:
        print(f"Found -> {patient.pid}, {patient.name}, {patient.age}, {patient.disease}, Priority:{patient.priority}")
    else:
        print("Patient not found.")
```

3. Delete Patient Record

- Patient record can be deleted using Patient ID.
- Deleted record is pushed into Stack for history.

```python
elif choice == '3':
    pid = input("Enter Patient ID to delete: ")
    patient = llist.delete(pid)
    if patient:
        deleted_stack.push(patient)
        print("Patient deleted successfully.")
    else:
        print("Patient not found.")
```

4. Display All Patient Records

- All stored patients are displayed sequentially using Linked List traversal.

```python
elif choice == '4':
    llist.display()
elif choice == '5':
    heap.display()
elif choice == '6':
    deleted_stack.display()
elif choice == '7':
    save_data(llist, heap)
elif choice == '8':
    save_data(llist, heap)
    print("Exiting system...")
    break
else:
    print("Invalid choice.")
```

5. Display Emergency Patients

- Emergency patients are displayed using Max Heap in level order.

```
def display(self):
    if not self.heap:
        print("No emergency patients.")
        return
    print("\nEmergency Patients (Max Heap - Level Order):")
    for p in self.heap:
        print(f"ID:{p.pid} Name:{p.name} Priority:{p.priority}")
```

6. Display Deleted Patients History

- Recently deleted patients are shown using Stack display.

```
def display(self):
    if not self.stack:
        print("No deleted patient history.")
        return
    print("\nDeleted Patients History:")
    for p in reversed(self.stack):
        print(f"ID:{p.pid} Name:{p.name}")
```

7. File Handling (Save and Load Data)

- All patient records are saved to a text file (patients.txt).
- When the program starts, previous data is automatically loaded.

```
def save_data(llist, heap):
    with open("patients.txt", "w") as f:
        temp = llist.head
        while temp:
            f.write(f"{temp.pid},{temp.name},{temp.age},{temp.disease},{temp.priority}\n")
            temp = temp.next
    print("Data saved successfully.")
```

8. Menu Driven Interface

A simple interactive menu allows the user to:

- Add patient
- Search patient
- Delete patient
- View all patients
- View emergency patients
- View deleted history
- Save data
- Exit system

```
--- Hospital Patient Management System ---
1. Add New Patient
2. Search Patient
3. Delete Patient
4. Display All Patients
5. Display Emergency Patients
6. Display Deleted Patients History
7. Save Data
8. Exit
```

5. Implementation Details

Programming Language

- Python

Data Structures Used

- Linked List
- Max Heap
- Stack

File Handling

- Text file used to store and load data persistently.

6. How Project Requirements Are Fulfilled

Requirement	Fulfillment
Use of at least 3 data structures	Implemented Linked List, Heap, and Stack
Insert operation	Implemented in Linked List and Heap
Search operation	Implemented using Linked List
Delete operation	Implemented in Linked List and stored in Stack
Display records	Sequential display (Linked List), Level order (Heap)

File handling	Save and load from text file
Menu-driven system	Fully implemented
Documentation & clarity	Properly commented code

7. Program Flow

1. The program starts and loads saved data from the file.
2. The main menu is displayed.
3. The user selects the desired option.
4. Operations are performed accordingly.
5. Data is saved before exit.

8. Expected Outputs

The system produces outputs such as:

- Patient added successfully
- Patient search results
- Display of all patients
- Display of emergency priority patients
- Deleted patient history
- Data saved and loaded messages

9. Advantages of the System

- Efficient data management
- Fast searching and deletion
- Priority-based emergency handling
- Permanent data storage
- Easy to use interface
- Real-world application of data structures

10. Code

```
class PatientNode:
    def __init__(self, pid, name, age, disease, priority):
        self.pid = pid
        self.name = name
        self.age = age
        self.disease = disease
```

```
        self.priority = priority  # emergency priority
        self.next = None

class PatientLinkedList:
    def __init__(self):
        self.head = None
    def insert(self, pid, name, age, disease, priority):
        new_node = PatientNode(pid, name, age, disease, priority)
        if self.head is None:
            self.head = new_node
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.next = new_node
    def search(self, pid):
        temp = self.head
        while temp:
            if temp.pid == pid:
                return temp
            temp = temp.next
        return None
    def delete(self, pid):
        temp = self.head
        prev = None
        while temp:
            if temp.pid == pid:
                if prev:
                    prev.next = temp.next
                else:
                    self.head = temp.next
                return temp
            prev = temp
            temp = temp.next
        return None
    def display(self):
        if self.head is None:
            print("No patient records found.")
            return
        temp = self.head
        print("\nPatient Records:")
        print("ID | Name | Age | Disease | Priority")
        print("-----------------------------------")
        while temp:
            print(f"{temp.pid} | {temp.name} | {temp.age} | {temp.disease} | {temp.priority}")
            temp = temp.next
```

```
    def to_list(self):
        data = []
        temp = self.head
        while temp:
            data.append(temp)
            temp = temp.next
        return data

class MaxHeap:
    def __init__(self):
        self.heap = []
    def insert(self, patient):
        self.heap.append(patient)
        self._heapify_up(len(self.heap) - 1)
    def _heapify_up(self, index):
        parent = (index - 1) // 2
        if index > 0 and self.heap[index].priority > self.heap[parent].priority:
            self.heap[index], self.heap[parent] = self.heap[parent], self.heap[index]
            self._heapify_up(parent)

    def delete_max(self):
        if not self.heap:
            return None
        if len(self.heap) == 1:
            return self.heap.pop()
        root = self.heap[0]
        self.heap[0] = self.heap.pop()
        self._heapify_down(0)
        return root
    def _heapify_down(self, index):
        largest = index
        left = 2 * index + 1
        right = 2 * index + 2
        if left < len(self.heap) and self.heap[left].priority > self.heap[largest].priority:
            largest = left
        if right < len(self.heap) and self.heap[right].priority > self.heap[largest].priority:
            largest = right
        if largest != index:
            self.heap[index], self.heap[largest] = self.heap[largest], self.heap[index]
            self._heapify_down(largest)
    def display(self):
        if not self.heap:
            print("No emergency patients.")
            return
        print("\nEmergency Patients (Max Heap - Level Order):")
        for p in self.heap:
```

```
        print(f"ID:{p.pid} Name:{p.name} Priority:{p.priority}")

class Stack:
    def __init__(self):
        self.stack = []
    def push(self, item):
        self.stack.append(item)
    def display(self):
        if not self.stack:
            print("No deleted patient history.")
            return
        print("\nDeleted Patients History:")
        for p in reversed(self.stack):
            print(f"ID:{p.pid} Name:{p.name}")
def save_data(llist, heap):
    with open("patients.txt", "w") as f:
        temp = llist.head
        while temp:
            f.write(f"{temp.pid},{temp.name},{temp.age},{temp.disease},{temp.priority}
")
            temp = temp.next
    print("Data saved successfully.")
def load_data(llist, heap):
    try:
        with open("patients.txt", "r") as f:
            for line in f:
                pid, name, age, disease, priority = line.strip().split(',')
                age = int(age)
                priority = int(priority)
                llist.insert(pid, name, age, disease, priority)
                heap.insert(PatientNode(pid, name, age, disease, priority))
        print("Data loaded successfully.")
    except FileNotFoundError:
        print("No saved data found.")
def main():
    llist = PatientLinkedList()
    heap = MaxHeap()
    deleted_stack = Stack()
    load_data(llist, heap)
    while True:
        print("\n--- Hospital Patient Management System ---")
        print("1. Add New Patient")
        print("2. Search Patient")
        print("3. Delete Patient")
        print("4. Display All Patients")
        print("5. Display Emergency Patients")
```

```
        print("6. Display Deleted Patients History")
        print("7. Save Data")
        print("8. Exit")
        choice = input("Enter choice: ")
        if choice == '1':
            pid = input("Patient ID: ")
            name = input("Name: ")
            age = int(input("Age: "))
            disease = input("Disease: ")
            priority = int(input("Emergency Priority (1-10): "))
            llist.insert(pid, name, age, disease, priority)
            heap.insert(PatientNode(pid, name, age, disease, priority))
            print("Patient added successfully.")
        elif choice == '2':
            pid = input("Enter Patient ID to search: ")
            patient = llist.search(pid)
            if patient:
                print(f"Found -> {patient.pid}, {patient.name}, {patient.age}, {patient.disease},
Priority:{patient.priority}")
            else:
                print("Patient not found.")
        elif choice == '3':
            pid = input("Enter Patient ID to delete: ")
            patient = llist.delete(pid)
            if patient:
                deleted_stack.push(patient)
                print("Patient deleted successfully.")
            else:
                print("Patient not found.")
        elif choice == '4':
            llist.display()
        elif choice == '5':
            heap.display()
        elif choice == '6':
            deleted_stack.display()
        elif choice == '7':
            save_data(llist, heap)
        elif choice == '8':
            save_data(llist, heap)
            print("Exiting system...")
            break
        else:
            print("Invalid choice.")
if __name__ == "__main__":
    main()
```

11. Output:

```
--- Hospital Patient Management System ---
1. Add New Patient
2. Search Patient
3. Delete Patient
4. Display All Patients
5. Display Emergency Patients
6. Display Deleted Patients History
7. Save Data
8. Exit
Enter choice: 1
Patient ID: 101
Name: Ali
Age: 23
Disease: Cancer
Emergency Priority (1-10): 4
Patient added successfully.

--- Hospital Patient Management System ---
1. Add New Patient
2. Search Patient
3. Delete Patient
4. Display All Patients
5. Display Emergency Patients
6. Display Deleted Patients History
7. Save Data
8. Exit
Enter choice: 1
Patient ID: 102
Name: Sara
Age: 21
Disease: Dengue Fever
Emergency Priority (1-10): 8
Patient added successfully.
```

```
--- Hospital Patient Management System ---
1. Add New Patient
2. Search Patient
3. Delete Patient
4. Display All Patients
5. Display Emergency Patients
6. Display Deleted Patients History
7. Save Data
8. Exit
Enter choice: 1
Patient ID: 103
Name: Mirza
Age: 45
Disease: Diabetes
Emergency Priority (1-10): 5
Patient added successfully.

--- Hospital Patient Management System ---
1. Add New Patient
2. Search Patient
3. Delete Patient
4. Display All Patients
5. Display Emergency Patients
6. Display Deleted Patients History
7. Save Data
8. Exit
Enter choice: 2
Enter Patient ID to search: 102
Found -> 102, Sara, 21, Dengue Fever, Priority:8
```

```
--- Hospital Patient Management System ---
1. Add New Patient
2. Search Patient
3. Delete Patient
4. Display All Patients
5. Display Emergency Patients
6. Display Deleted Patients History
7. Save Data
8. Exit
Enter choice: 4

Patient Records:
ID | Name | Age | Disease | Priority
----------------------------------
101 | Ali | 23 | Cancer | 4
102 | Sara | 21 | Dengue Fever | 8
103 | Mirza | 45 | Diabetes | 5

--- Hospital Patient Management System ---
1. Add New Patient
2. Search Patient
3. Delete Patient
4. Display All Patients
5. Display Emergency Patients
6. Display Deleted Patients History
7. Save Data
8. Exit
Enter choice: 5

Emergency Patients (Max Heap - Level Order):
ID:102 Name:Sara Priority:8
ID:101 Name:Ali Priority:4
ID:103 Name:Mirza Priority:5
```

```
--- Hospital Patient Management System ---
1. Add New Patient
2. Search Patient
3. Delete Patient
4. Display All Patients
5. Display Emergency Patients
6. Display Deleted Patients History
7. Save Data
8. Exit
Enter choice: 7
Data saved successfully.

--- Hospital Patient Management System ---
1. Add New Patient
2. Search Patient
3. Delete Patient
4. Display All Patients
5. Display Emergency Patients
6. Display Deleted Patients History
7. Save Data
8. Exit
Enter choice: 3
Enter Patient ID to delete: 102
Patient deleted successfully.

--- Hospital Patient Management System ---
1. Add New Patient
2. Search Patient
3. Delete Patient
4. Display All Patients
5. Display Emergency Patients
6. Display Deleted Patients History
7. Save Data
8. Exit
Enter choice: 4
```

```
Patient Records:
ID | Name | Age | Disease | Priority
----------------------------------
101 | Ali | 23 | Cancer | 4
103 | Mirza | 45 | Diabetes | 5
```

12. Conclusion

This project successfully implements a Hospital Management System using fundamental data structures. The use of Linked List, Max Heap, and Stack provides an efficient and organized way to manage patient information. All required operations such as insertion, searching, deletion, display, file handling, and menu-driven interaction have been implemented properly. The project demonstrates the practical use of data structures in solving real-world problems and helps in understanding their importance in software development.